exercise1

March 29, 2026

1 Exercise 1

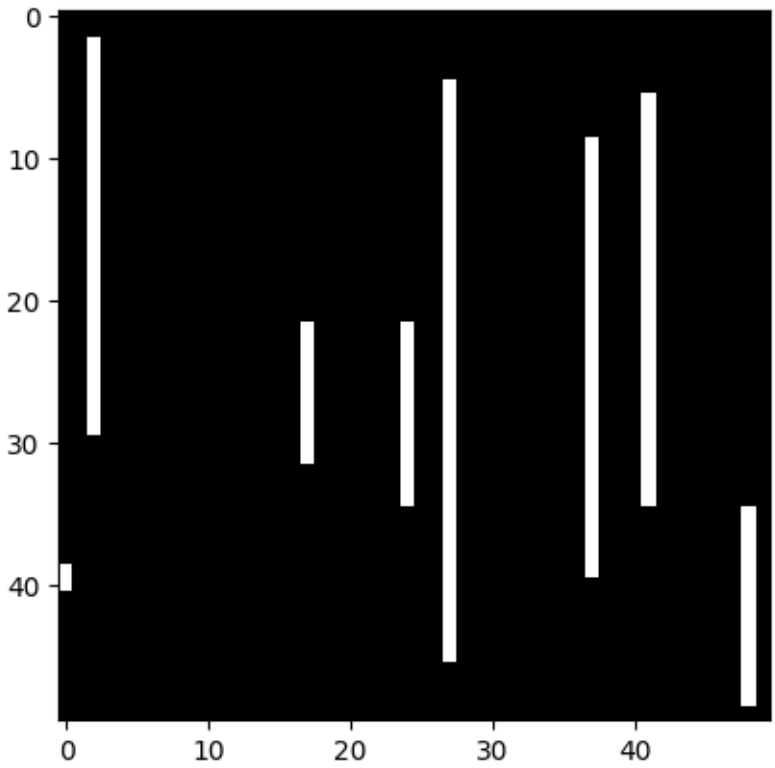
[1]:
```
import numpy as np
```

[2]:
```
def generate_stripe_image(size, stripe_nr, vertical = True):
  img=np.zeros((size,size,1),dtype="uint8")
  for i in range(0,stripe_nr):
    x,y = np.random.randint(0,size,2)
    l  = np.random.randint(y,size,1).item()
    if (vertical):
      img[y:l,x,0]=255
    else:
      img[x,y:l,0]=255
  return img
```

[3]:
```
import matplotlib.pyplot as plt
import numpy as np

img=generate_stripe_image(50,10, vertical=True)
plt.imshow(img[:,:,0],cmap='gray')
```

[3]:
```
<matplotlib.image.AxesImage at 0x10bfd9220>
```



2 1.1

[4]:
```
size = 50
stripe_nr = 10
X_train = np.stack(
    [generate_stripe_image(size, stripe_nr, vertical=True) for _ in range(500)]␣
 ↪+
    [generate_stripe_image(size, stripe_nr, vertical=False) for _ in range(500)]
)  / 255
X_val = np.stack(
    [generate_stripe_image(size, stripe_nr, vertical=True) for _ in range(500)]␣
 ↪+
    [generate_stripe_image(size, stripe_nr, vertical=False) for _ in range(500)]
)  / 255
```

3 1.2

[5]:
```
from tensorflow.keras.utils import to_categorical
Y_train = np.array([0 for _ in range(500)] + [1 for _ in range(500)])
Y_val = np.array([0 for _ in range(500)] + [1 for _ in range(500)])
```

```
Y_train, Y_val = to_categorical(Y_train), to_categorical(Y_val)
```

4 1.3

[18]:
```
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential

# Model
model = Sequential()

model.add(layers.Conv2D(filters=1, kernel_size=(5,5), activation='relu',␣
 ↪input_shape=(50,50,1)))
model.add(layers.MaxPooling2D(pool_size=(50,50), padding='same'))
model.add(layers.Flatten())
model.add(layers.Dense(2, activation='softmax'))
model.summary()
```

```
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
conv2d_5 (Conv2D)	(None, 46, 46, 1)	26
max_pooling2d_4 (MaxPooling2D)	(None, 1, 1, 1)	0
flatten_3 (Flatten)	(None, 1)	0
dense_3 (Dense)	(None, 2)	4

```
Total params: 30 (120.00 B)

Trainable params: 30 (120.00 B)

Non-trainable params: 0 (0.00 B)
```

[ ]:
```
# Compile
model.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])
```

5 1.4

[ ]:
```
history=model.fit(X_train, Y_train,
                  validation_data=(X_val,Y_val),
                  batch_size=64,
                  epochs=50,
                  verbose=1,
                  shuffle=True)
```

```
Epoch 1/50
16/16             1s 25ms/step -
accuracy: 0.5000 - loss: 0.7262 - val_accuracy: 0.5000 - val_loss: 0.7137
Epoch 2/50
16/16             0s 11ms/step -
accuracy: 0.5000 - loss: 0.7038 - val_accuracy: 0.5000 - val_loss: 0.6949
Epoch 3/50
16/16             0s 11ms/step -
accuracy: 0.5000 - loss: 0.6869 - val_accuracy: 0.5000 - val_loss: 0.6797
Epoch 4/50
16/16             0s 12ms/step -
accuracy: 0.5000 - loss: 0.6731 - val_accuracy: 0.5000 - val_loss: 0.6673
Epoch 5/50
16/16             0s 12ms/step -
accuracy: 0.5000 - loss: 0.6613 - val_accuracy: 0.5000 - val_loss: 0.6562
Epoch 6/50
16/16             0s 12ms/step -
accuracy: 0.5000 - loss: 0.6502 - val_accuracy: 0.5000 - val_loss: 0.6450
Epoch 7/50
16/16             0s 12ms/step -
accuracy: 0.5000 - loss: 0.6385 - val_accuracy: 0.5000 - val_loss: 0.6328
Epoch 8/50
16/16             0s 11ms/step -
accuracy: 0.5280 - loss: 0.6255 - val_accuracy: 0.5370 - val_loss: 0.6189
Epoch 9/50
16/16             0s 12ms/step -
accuracy: 0.5420 - loss: 0.6107 - val_accuracy: 0.5510 - val_loss: 0.6031
Epoch 10/50
16/16             0s 10ms/step -
accuracy: 0.5610 - loss: 0.5943 - val_accuracy: 0.5760 - val_loss: 0.5858
Epoch 11/50
16/16             0s 12ms/step -
accuracy: 0.6220 - loss: 0.5766 - val_accuracy: 0.6480 - val_loss: 0.5672
Epoch 12/50
16/16             0s 11ms/step -
accuracy: 0.6960 - loss: 0.5577 - val_accuracy: 0.7730 - val_loss: 0.5475
Epoch 13/50
16/16             0s 10ms/step -
accuracy: 0.8080 - loss: 0.5381 - val_accuracy: 0.8890 - val_loss: 0.5277
```

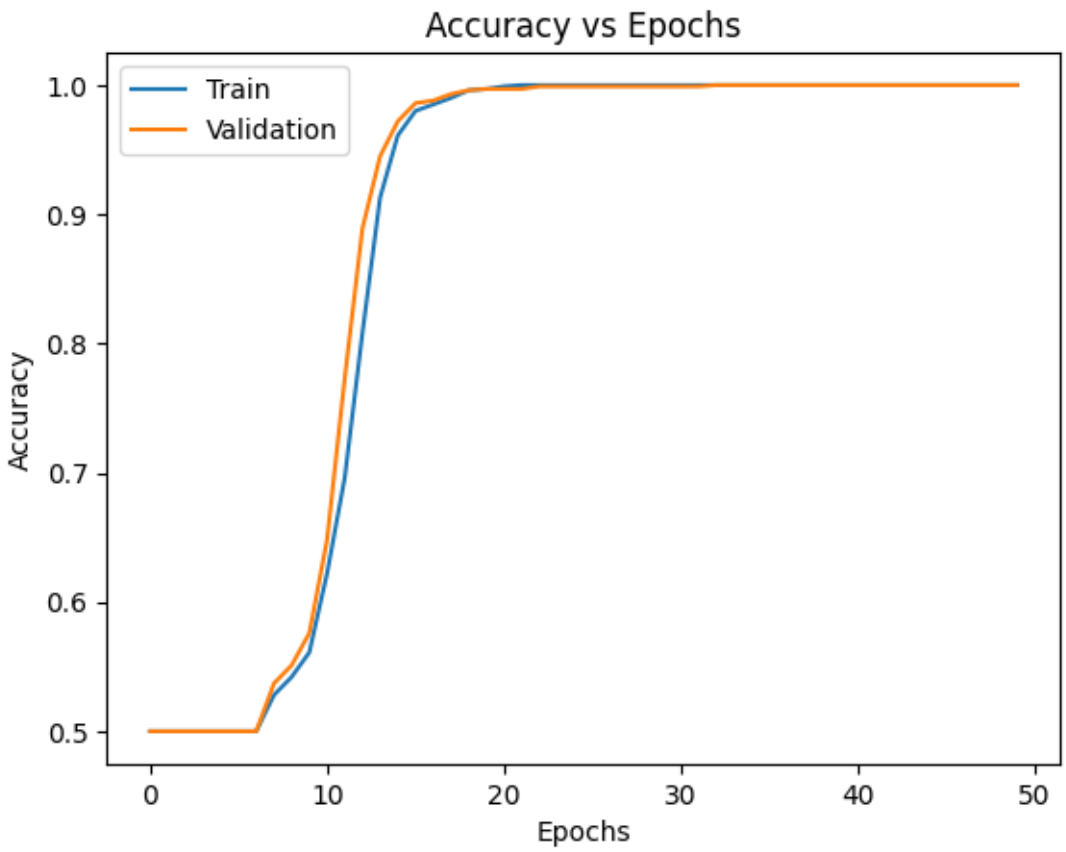
```
Epoch 14/50
16/16             0s 10ms/step -
accuracy: 0.9130 - loss: 0.5182 - val_accuracy: 0.9450 - val_loss: 0.5076
Epoch 15/50
16/16             0s 11ms/step -
accuracy: 0.9610 - loss: 0.4980 - val_accuracy: 0.9720 - val_loss: 0.4876
Epoch 16/50
16/16             0s 10ms/step -
accuracy: 0.9800 - loss: 0.4782 - val_accuracy: 0.9860 - val_loss: 0.4679
Epoch 17/50
16/16             0s 10ms/step -
accuracy: 0.9850 - loss: 0.4587 - val_accuracy: 0.9880 - val_loss: 0.4485
Epoch 18/50
16/16             0s 11ms/step -
accuracy: 0.9900 - loss: 0.4395 - val_accuracy: 0.9930 - val_loss: 0.4296
Epoch 19/50
16/16             0s 11ms/step -
accuracy: 0.9960 - loss: 0.4210 - val_accuracy: 0.9960 - val_loss: 0.4111
Epoch 20/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.4023 - val_accuracy: 0.9970 - val_loss: 0.3927
Epoch 21/50
16/16             0s 15ms/step -
accuracy: 0.9990 - loss: 0.3841 - val_accuracy: 0.9970 - val_loss: 0.3748
Epoch 22/50
16/16             0s 13ms/step -
accuracy: 1.0000 - loss: 0.3665 - val_accuracy: 0.9970 - val_loss: 0.3576
Epoch 23/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.3499 - val_accuracy: 0.9990 - val_loss: 0.3418
Epoch 24/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.3343 - val_accuracy: 0.9990 - val_loss: 0.3267
Epoch 25/50
16/16             0s 12ms/step -
accuracy: 1.0000 - loss: 0.3199 - val_accuracy: 0.9990 - val_loss: 0.3130
Epoch 26/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.3065 - val_accuracy: 0.9990 - val_loss: 0.3002
Epoch 27/50
16/16             0s 14ms/step -
accuracy: 1.0000 - loss: 0.2941 - val_accuracy: 0.9990 - val_loss: 0.2881
Epoch 28/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2823 - val_accuracy: 0.9990 - val_loss: 0.2768
Epoch 29/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2713 - val_accuracy: 0.9990 - val_loss: 0.2662
```

```
Epoch 30/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2610 - val_accuracy: 0.9990 - val_loss: 0.2562
Epoch 31/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2512 - val_accuracy: 0.9990 - val_loss: 0.2466
Epoch 32/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2419 - val_accuracy: 0.9990 - val_loss: 0.2376
Epoch 33/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2331 - val_accuracy: 1.0000 - val_loss: 0.2290
Epoch 34/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2247 - val_accuracy: 1.0000 - val_loss: 0.2208
Epoch 35/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.2167 - val_accuracy: 1.0000 - val_loss: 0.2129
Epoch 36/50
16/16             0s 12ms/step -
accuracy: 1.0000 - loss: 0.2090 - val_accuracy: 1.0000 - val_loss: 0.2054
Epoch 37/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.2017 - val_accuracy: 1.0000 - val_loss: 0.1983
Epoch 38/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1948 - val_accuracy: 1.0000 - val_loss: 0.1915
Epoch 39/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1883 - val_accuracy: 1.0000 - val_loss: 0.1850
Epoch 40/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1820 - val_accuracy: 1.0000 - val_loss: 0.1789
Epoch 41/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1761 - val_accuracy: 1.0000 - val_loss: 0.1732
Epoch 42/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1705 - val_accuracy: 1.0000 - val_loss: 0.1677
Epoch 43/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1653 - val_accuracy: 1.0000 - val_loss: 0.1625
Epoch 44/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1602 - val_accuracy: 1.0000 - val_loss: 0.1577
Epoch 45/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1555 - val_accuracy: 1.0000 - val_loss: 0.1531
```

```
Epoch 46/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1511 - val_accuracy: 1.0000 - val_loss: 0.1488
Epoch 47/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1468 - val_accuracy: 1.0000 - val_loss: 0.1447
Epoch 48/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1428 - val_accuracy: 1.0000 - val_loss: 0.1408
Epoch 49/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1390 - val_accuracy: 1.0000 - val_loss: 0.1371
Epoch 50/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1353 - val_accuracy: 1.0000 - val_loss: 0.1335
```

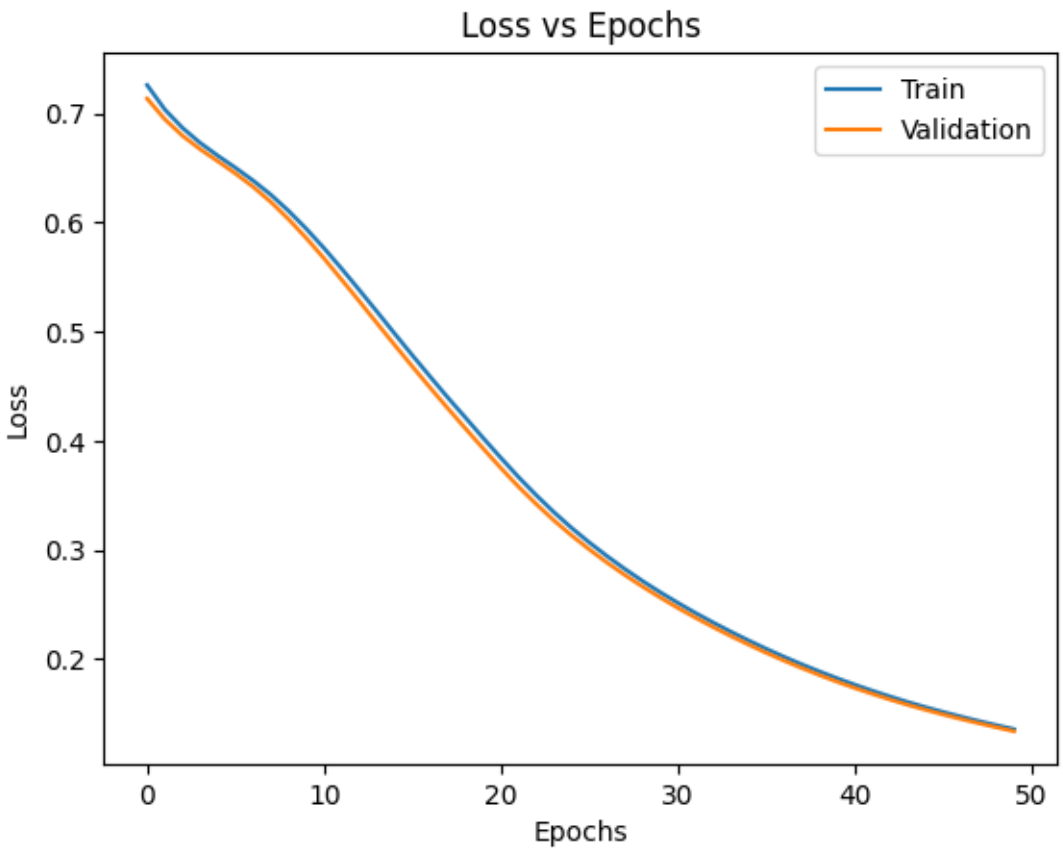
[9]:
```
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()
```



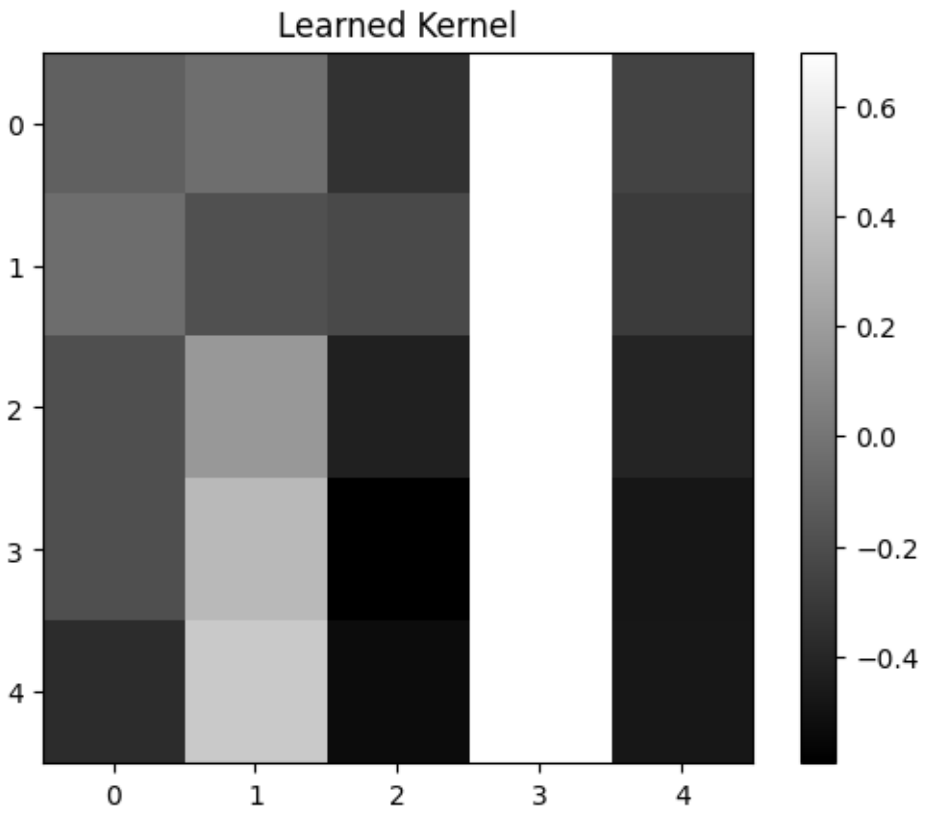
[10]:
```
loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```



6 1.5

[11]:
```
conv_filter=model.get_weights()[0]
my_kernel = conv_filter[:, :, 0, 0]
plt.imshow(my_kernel, cmap='gray')
plt.colorbar()
plt.title("Learned Kernel")
plt.show()
```



The kernel is made of 5x5 grayscale pixels. It doesen't seem reasonable for the kernel to distinguish horizontal and vertical stripes because there is no appearent pattern in the kernel.

7 1.6

[14]:
```
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential

# Model
model_relu = Sequential()

model_relu.add(layers.Conv2D(filters=1, kernel_size=(5,5), activation='relu',␣
↪input_shape=(50,50,1)))
model_relu.add(layers.MaxPooling2D(pool_size=(50,50), padding='same'))
model_relu.add(layers.Flatten())
model_relu.add(layers.Dense(2, activation='softmax'))
model_relu.summary()

# Compile
```

```
model_relu.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])

history=model_relu.fit(X_train, Y_train,
                  validation_data=(X_val,Y_val),
                  batch_size=64,
                  epochs=50,
                  verbose=1,
                  shuffle=True)

accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
plt.title("Accuracy vs Epochs model_relu")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs model_relu")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 46, 46, 1)	26
max_pooling2d_3 (MaxPooling2D)	(None, 1, 1, 1)	0
flatten_1 (Flatten)	(None, 1)	0
dense_1 (Dense)	(None, 2)	4

```
 Total params: 30 (120.00 B)

 Trainable params: 30 (120.00 B)

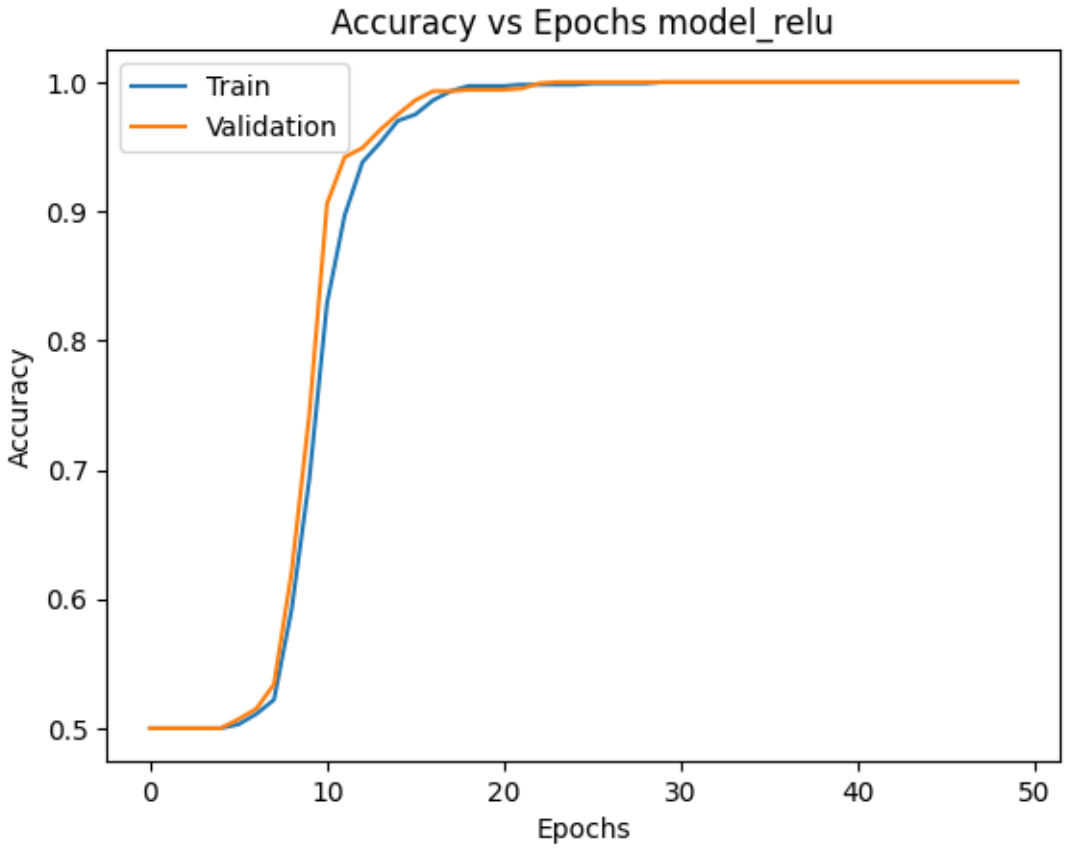
 Non-trainable params: 0 (0.00 B)

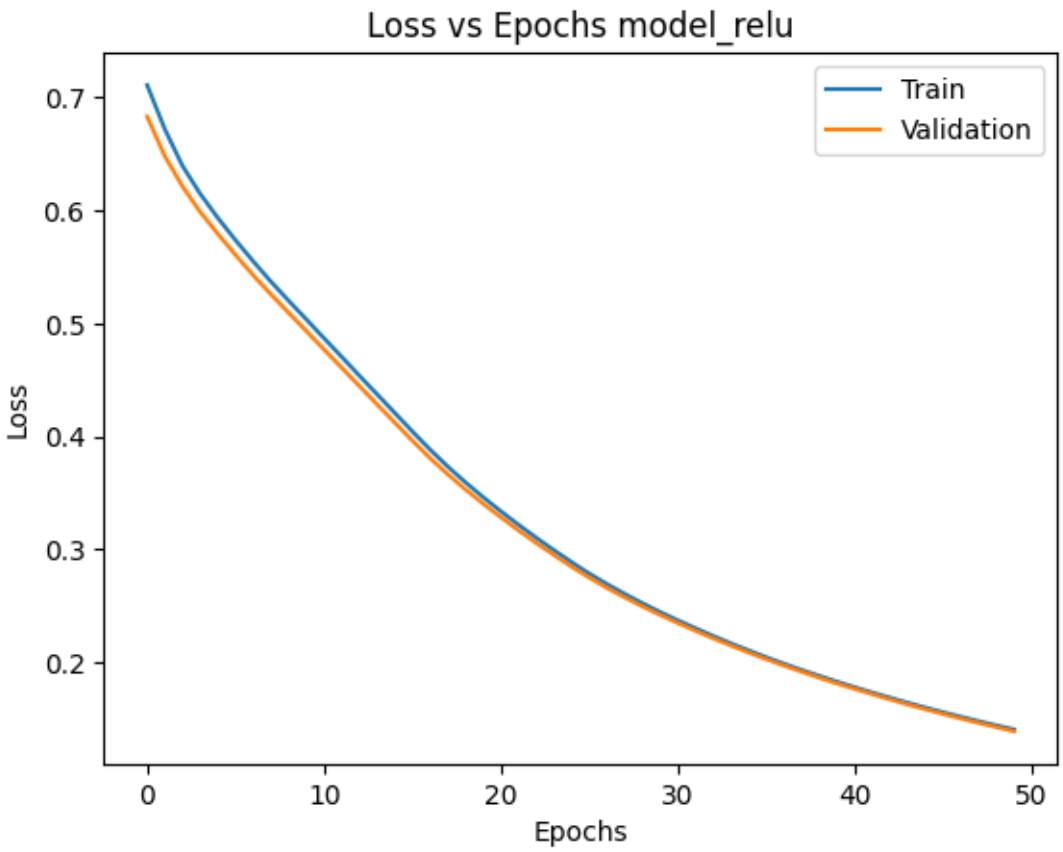
Epoch 1/50
16/16             1s 21ms/step -
accuracy: 0.5000 - loss: 0.7111 - val_accuracy: 0.5000 - val_loss: 0.6832
Epoch 2/50
16/16             0s 15ms/step -
accuracy: 0.5000 - loss: 0.6721 - val_accuracy: 0.5000 - val_loss: 0.6483
Epoch 3/50
16/16             0s 13ms/step -
accuracy: 0.5000 - loss: 0.6396 - val_accuracy: 0.5000 - val_loss: 0.6214
Epoch 4/50
16/16             0s 11ms/step -
accuracy: 0.5000 - loss: 0.6149 - val_accuracy: 0.5000 - val_loss: 0.5989
Epoch 5/50
16/16             0s 12ms/step -
accuracy: 0.5000 - loss: 0.5935 - val_accuracy: 0.5000 - val_loss: 0.5794
Epoch 6/50
16/16             0s 13ms/step -
accuracy: 0.5030 - loss: 0.5738 - val_accuracy: 0.5070 - val_loss: 0.5608
Epoch 7/50
16/16             0s 13ms/step -
accuracy: 0.5110 - loss: 0.5551 - val_accuracy: 0.5150 - val_loss: 0.5427
Epoch 8/50
16/16             0s 16ms/step -
accuracy: 0.5220 - loss: 0.5371 - val_accuracy: 0.5340 - val_loss: 0.5260
Epoch 9/50
16/16             0s 12ms/step -
accuracy: 0.5920 - loss: 0.5203 - val_accuracy: 0.6210 - val_loss: 0.5096
Epoch 10/50
16/16             0s 12ms/step -
accuracy: 0.6930 - loss: 0.5037 - val_accuracy: 0.7430 - val_loss: 0.4934
Epoch 11/50
16/16             0s 11ms/step -
accuracy: 0.8290 - loss: 0.4871 - val_accuracy: 0.9060 - val_loss: 0.4771
Epoch 12/50
16/16             0s 11ms/step -
accuracy: 0.8970 - loss: 0.4705 - val_accuracy: 0.9420 - val_loss: 0.4609
Epoch 13/50
16/16             0s 11ms/step -
accuracy: 0.9380 - loss: 0.4539 - val_accuracy: 0.9490 - val_loss: 0.4447
```

```
Epoch 14/50
16/16             0s 11ms/step -
accuracy: 0.9530 - loss: 0.4375 - val_accuracy: 0.9630 - val_loss: 0.4285
Epoch 15/50
16/16             0s 11ms/step -
accuracy: 0.9700 - loss: 0.4210 - val_accuracy: 0.9750 - val_loss: 0.4124
Epoch 16/50
16/16             0s 11ms/step -
accuracy: 0.9750 - loss: 0.4045 - val_accuracy: 0.9860 - val_loss: 0.3963
Epoch 17/50
16/16             0s 11ms/step -
accuracy: 0.9860 - loss: 0.3884 - val_accuracy: 0.9930 - val_loss: 0.3809
Epoch 18/50
16/16             0s 11ms/step -
accuracy: 0.9930 - loss: 0.3734 - val_accuracy: 0.9930 - val_loss: 0.3666
Epoch 19/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.3595 - val_accuracy: 0.9940 - val_loss: 0.3532
Epoch 20/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.3463 - val_accuracy: 0.9940 - val_loss: 0.3406
Epoch 21/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.3338 - val_accuracy: 0.9940 - val_loss: 0.3286
Epoch 22/50
16/16             0s 13ms/step -
accuracy: 0.9980 - loss: 0.3219 - val_accuracy: 0.9950 - val_loss: 0.3170
Epoch 23/50
16/16             0s 11ms/step -
accuracy: 0.9980 - loss: 0.3104 - val_accuracy: 0.9990 - val_loss: 0.3058
Epoch 24/50
16/16             0s 12ms/step -
accuracy: 0.9980 - loss: 0.2994 - val_accuracy: 1.0000 - val_loss: 0.2951
Epoch 25/50
16/16             0s 11ms/step -
accuracy: 0.9980 - loss: 0.2888 - val_accuracy: 1.0000 - val_loss: 0.2848
Epoch 26/50
16/16             0s 10ms/step -
accuracy: 0.9990 - loss: 0.2787 - val_accuracy: 1.0000 - val_loss: 0.2751
Epoch 27/50
16/16             0s 11ms/step -
accuracy: 0.9990 - loss: 0.2694 - val_accuracy: 1.0000 - val_loss: 0.2661
Epoch 28/50
16/16             0s 11ms/step -
accuracy: 0.9990 - loss: 0.2607 - val_accuracy: 1.0000 - val_loss: 0.2577
Epoch 29/50
16/16             0s 11ms/step -
accuracy: 0.9990 - loss: 0.2525 - val_accuracy: 1.0000 - val_loss: 0.2497
```

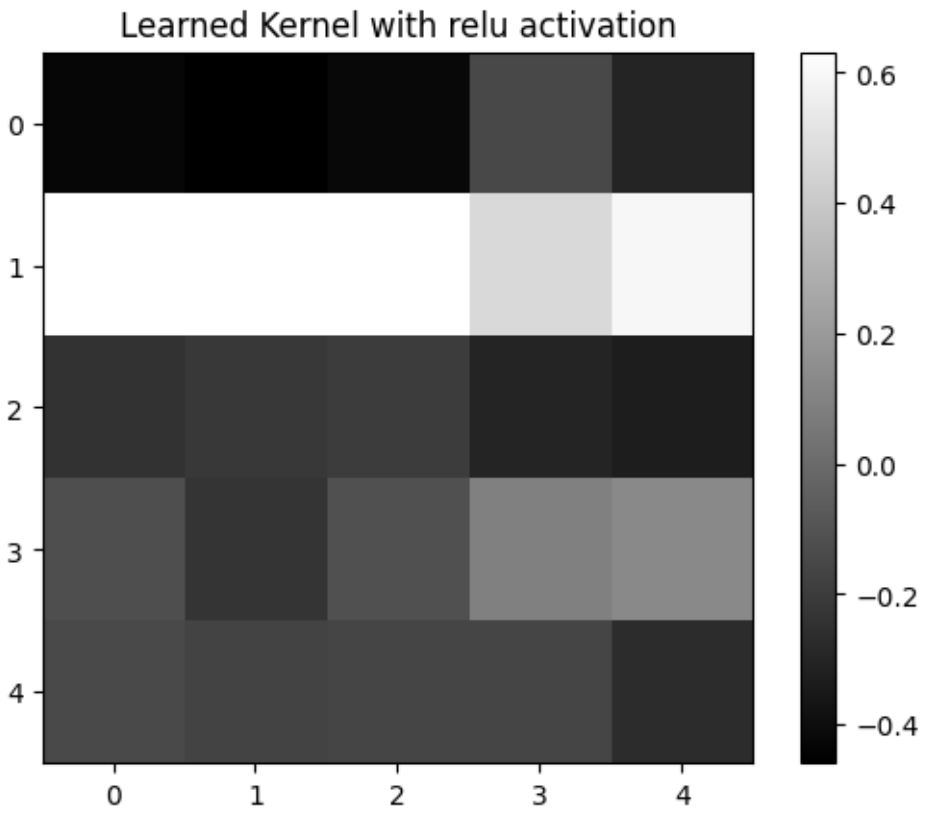
```
Epoch 30/50
16/16             0s 13ms/step -
accuracy: 1.0000 - loss: 0.2447 - val_accuracy: 1.0000 - val_loss: 0.2422
Epoch 31/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.2373 - val_accuracy: 1.0000 - val_loss: 0.2351
Epoch 32/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.2303 - val_accuracy: 1.0000 - val_loss: 0.2281
Epoch 33/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.2235 - val_accuracy: 1.0000 - val_loss: 0.2215
Epoch 34/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.2170 - val_accuracy: 1.0000 - val_loss: 0.2151
Epoch 35/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.2107 - val_accuracy: 1.0000 - val_loss: 0.2089
Epoch 36/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.2047 - val_accuracy: 1.0000 - val_loss: 0.2030
Epoch 37/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1990 - val_accuracy: 1.0000 - val_loss: 0.1973
Epoch 38/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1934 - val_accuracy: 1.0000 - val_loss: 0.1919
Epoch 39/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1882 - val_accuracy: 1.0000 - val_loss: 0.1866
Epoch 40/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1831 - val_accuracy: 1.0000 - val_loss: 0.1815
Epoch 41/50
16/16             0s 13ms/step -
accuracy: 1.0000 - loss: 0.1781 - val_accuracy: 1.0000 - val_loss: 0.1767
Epoch 42/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1734 - val_accuracy: 1.0000 - val_loss: 0.1720
Epoch 43/50
16/16             0s 12ms/step -
accuracy: 1.0000 - loss: 0.1689 - val_accuracy: 1.0000 - val_loss: 0.1674
Epoch 44/50
16/16             0s 12ms/step -
accuracy: 1.0000 - loss: 0.1644 - val_accuracy: 1.0000 - val_loss: 0.1629
Epoch 45/50
16/16             0s 12ms/step -
accuracy: 1.0000 - loss: 0.1601 - val_accuracy: 1.0000 - val_loss: 0.1586
```

```
Epoch 46/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1560 - val_accuracy: 1.0000 - val_loss: 0.1545
Epoch 47/50
16/16             0s 14ms/step -
accuracy: 1.0000 - loss: 0.1520 - val_accuracy: 1.0000 - val_loss: 0.1505
Epoch 48/50
16/16             0s 10ms/step -
accuracy: 1.0000 - loss: 0.1481 - val_accuracy: 1.0000 - val_loss: 0.1466
Epoch 49/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1444 - val_accuracy: 1.0000 - val_loss: 0.1429
Epoch 50/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.1408 - val_accuracy: 1.0000 - val_loss: 0.1392
```





[15]:
```
conv_filter=model_relu.get_weights()[0]
my_kernel = conv_filter[:, :, 0, 0]
plt.imshow(my_kernel, cmap='gray')
plt.colorbar()
plt.title("Learned Kernel with relu activation")
plt.show()
```



With relu activation, we have the same accuracy of 1 on the validation set but the kernel's appearence changed.

[16]:
```
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential

# Model
mode_avg = Sequential()

mode_avg.add(layers.Conv2D(filters=1, kernel_size=(5,5), activation='relu',␣
 ↪input_shape=(50,50,1)), )
mode_avg.add(layers.AveragePooling2D(pool_size=(50,50), padding='same'))
mode_avg.add(layers.Flatten())
mode_avg.add(layers.Dense(2, activation='softmax'))
mode_avg.summary()

# Compile
mode_avg.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])

history=mode_avg.fit(X_train, Y_train,
```

```
                    validation_data=(X_val,Y_val),
                    batch_size=64,
                    epochs=50,
                    verbose=1,
                    shuffle=True)

accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
plt.title("Accuracy vs Epochs mode_avg")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs mode_avg")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

```
/opt/anaconda3/lib/python3.12/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 46, 46, 1)	26
average_pooling2d (AveragePooling2D)	(None, 1, 1, 1)	0
flatten_2 (Flatten)	(None, 1)	0

```
 dense_2 (Dense)                 (None, 2)                      4


 Total params: 30 (120.00 B)

 Trainable params: 30 (120.00 B)

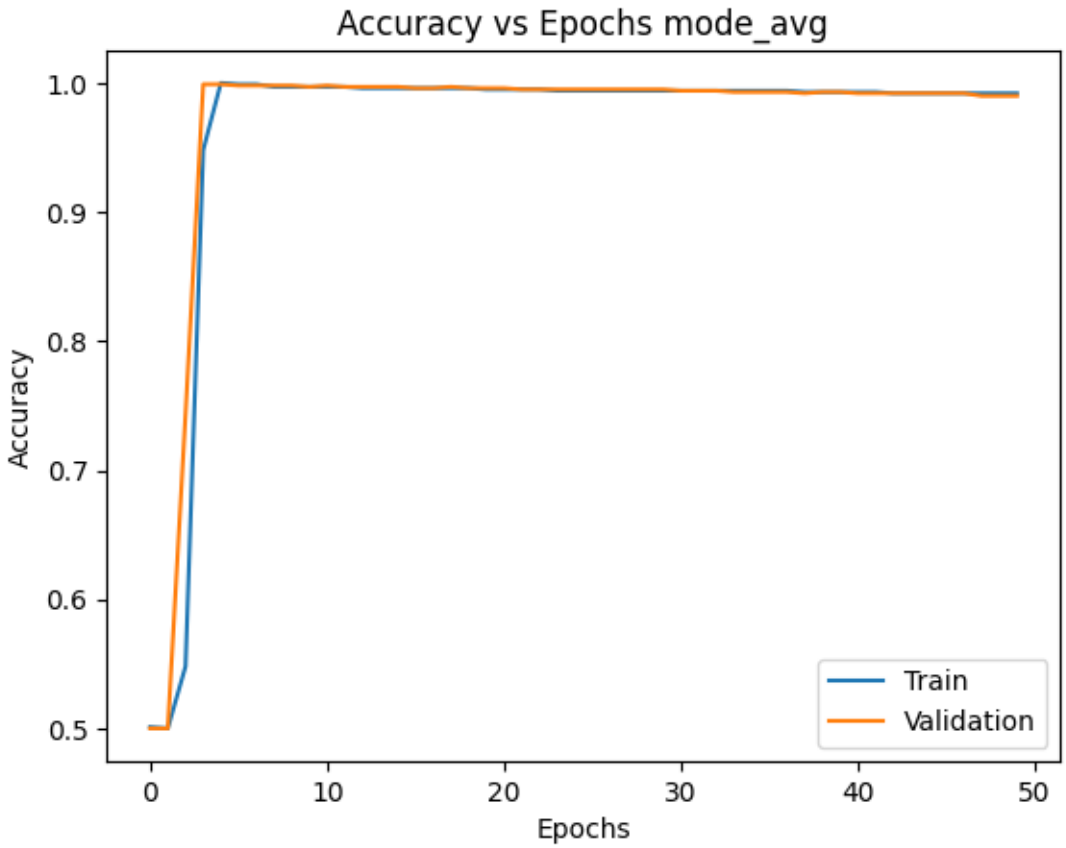
 Non-trainable params: 0 (0.00 B)

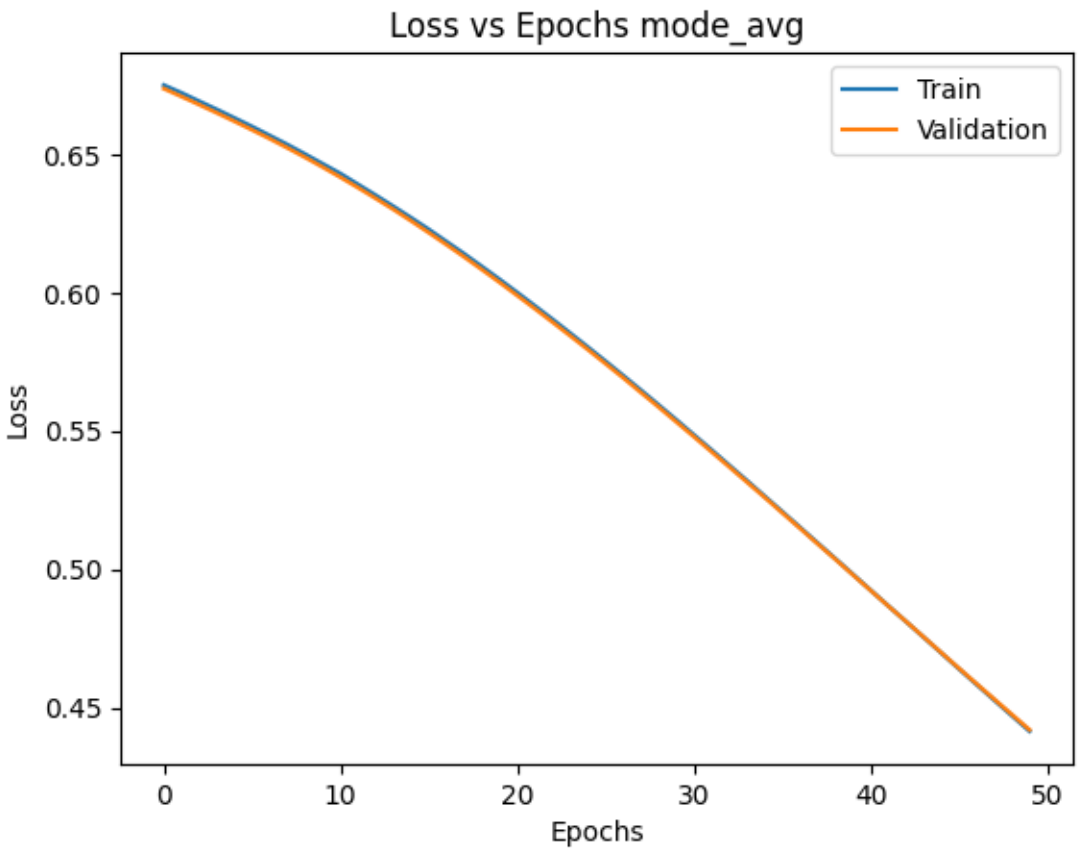
Epoch 1/50
16/16             1s 18ms/step -
accuracy: 0.5010 - loss: 0.6750 - val_accuracy: 0.5000 - val_loss: 0.6738
Epoch 2/50
16/16             0s 11ms/step -
accuracy: 0.5000 - loss: 0.6722 - val_accuracy: 0.5000 - val_loss: 0.6709
Epoch 3/50
16/16             0s 11ms/step -
accuracy: 0.5480 - loss: 0.6692 - val_accuracy: 0.7430 - val_loss: 0.6680
Epoch 4/50
16/16             0s 11ms/step -
accuracy: 0.9480 - loss: 0.6662 - val_accuracy: 0.9990 - val_loss: 0.6650
Epoch 5/50
16/16             0s 11ms/step -
accuracy: 1.0000 - loss: 0.6631 - val_accuracy: 0.9990 - val_loss: 0.6619
Epoch 6/50
16/16             0s 11ms/step -
accuracy: 0.9990 - loss: 0.6600 - val_accuracy: 0.9980 - val_loss: 0.6588
Epoch 7/50
16/16             0s 11ms/step -
accuracy: 0.9990 - loss: 0.6568 - val_accuracy: 0.9980 - val_loss: 0.6556
Epoch 8/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.6535 - val_accuracy: 0.9980 - val_loss: 0.6523
Epoch 9/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.6501 - val_accuracy: 0.9980 - val_loss: 0.6489
Epoch 10/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.6466 - val_accuracy: 0.9970 - val_loss: 0.6454
Epoch 11/50
16/16             0s 12ms/step -
accuracy: 0.9970 - loss: 0.6430 - val_accuracy: 0.9980 - val_loss: 0.6417
Epoch 12/50
16/16             0s 11ms/step -
accuracy: 0.9970 - loss: 0.6391 - val_accuracy: 0.9970 - val_loss: 0.6379
```

```
Epoch 13/50
16/16             0s 12ms/step -
accuracy: 0.9960 - loss: 0.6352 - val_accuracy: 0.9970 - val_loss: 0.6340
Epoch 14/50
16/16             0s 11ms/step -
accuracy: 0.9960 - loss: 0.6312 - val_accuracy: 0.9970 - val_loss: 0.6301
Epoch 15/50
16/16             0s 11ms/step -
accuracy: 0.9960 - loss: 0.6271 - val_accuracy: 0.9970 - val_loss: 0.6259
Epoch 16/50
16/16             0s 10ms/step -
accuracy: 0.9960 - loss: 0.6229 - val_accuracy: 0.9960 - val_loss: 0.6217
Epoch 17/50
16/16             0s 11ms/step -
accuracy: 0.9960 - loss: 0.6185 - val_accuracy: 0.9960 - val_loss: 0.6173
Epoch 18/50
16/16             0s 13ms/step -
accuracy: 0.9960 - loss: 0.6141 - val_accuracy: 0.9970 - val_loss: 0.6129
Epoch 19/50
16/16             0s 11ms/step -
accuracy: 0.9960 - loss: 0.6095 - val_accuracy: 0.9960 - val_loss: 0.6084
Epoch 20/50
16/16             0s 11ms/step -
accuracy: 0.9950 - loss: 0.6048 - val_accuracy: 0.9960 - val_loss: 0.6037
Epoch 21/50
16/16             0s 11ms/step -
accuracy: 0.9950 - loss: 0.6001 - val_accuracy: 0.9960 - val_loss: 0.5990
Epoch 22/50
16/16             0s 10ms/step -
accuracy: 0.9950 - loss: 0.5952 - val_accuracy: 0.9950 - val_loss: 0.5942
Epoch 23/50
16/16             0s 10ms/step -
accuracy: 0.9950 - loss: 0.5904 - val_accuracy: 0.9950 - val_loss: 0.5893
Epoch 24/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5854 - val_accuracy: 0.9950 - val_loss: 0.5844
Epoch 25/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5803 - val_accuracy: 0.9950 - val_loss: 0.5794
Epoch 26/50
16/16             0s 11ms/step -
accuracy: 0.9940 - loss: 0.5752 - val_accuracy: 0.9950 - val_loss: 0.5743
Epoch 27/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5700 - val_accuracy: 0.9950 - val_loss: 0.5692
Epoch 28/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5648 - val_accuracy: 0.9950 - val_loss: 0.5639
```

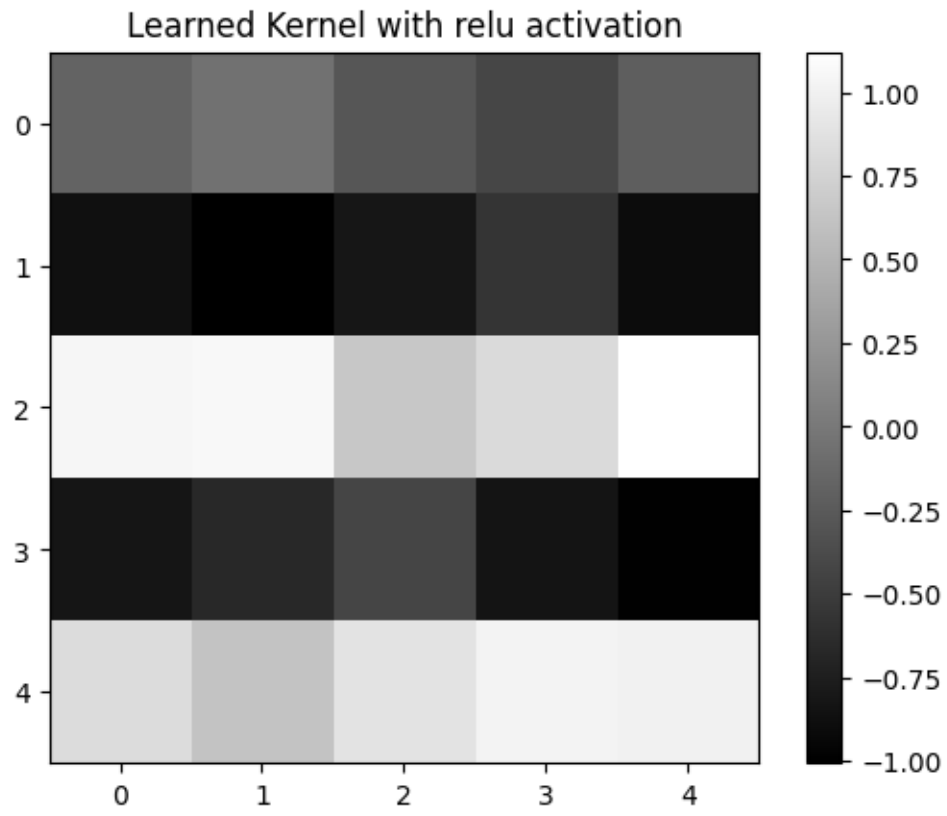
```
Epoch 29/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5594 - val_accuracy: 0.9950 - val_loss: 0.5587
Epoch 30/50
16/16             0s 9ms/step -
accuracy: 0.9940 - loss: 0.5541 - val_accuracy: 0.9950 - val_loss: 0.5533
Epoch 31/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5486 - val_accuracy: 0.9940 - val_loss: 0.5479
Epoch 32/50
16/16             0s 9ms/step -
accuracy: 0.9940 - loss: 0.5432 - val_accuracy: 0.9940 - val_loss: 0.5425
Epoch 33/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5376 - val_accuracy: 0.9940 - val_loss: 0.5370
Epoch 34/50
16/16             0s 13ms/step -
accuracy: 0.9940 - loss: 0.5321 - val_accuracy: 0.9930 - val_loss: 0.5315
Epoch 35/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5265 - val_accuracy: 0.9930 - val_loss: 0.5260
Epoch 36/50
16/16             0s 10ms/step -
accuracy: 0.9940 - loss: 0.5209 - val_accuracy: 0.9930 - val_loss: 0.5204
Epoch 37/50
16/16             0s 11ms/step -
accuracy: 0.9940 - loss: 0.5152 - val_accuracy: 0.9930 - val_loss: 0.5149
Epoch 38/50
16/16             0s 11ms/step -
accuracy: 0.9930 - loss: 0.5095 - val_accuracy: 0.9920 - val_loss: 0.5092
Epoch 39/50
16/16             0s 11ms/step -
accuracy: 0.9930 - loss: 0.5039 - val_accuracy: 0.9930 - val_loss: 0.5037
Epoch 40/50
16/16             0s 11ms/step -
accuracy: 0.9930 - loss: 0.4982 - val_accuracy: 0.9930 - val_loss: 0.4980
Epoch 41/50
16/16             0s 11ms/step -
accuracy: 0.9930 - loss: 0.4925 - val_accuracy: 0.9920 - val_loss: 0.4924
Epoch 42/50
16/16             0s 12ms/step -
accuracy: 0.9930 - loss: 0.4868 - val_accuracy: 0.9920 - val_loss: 0.4867
Epoch 43/50
16/16             0s 10ms/step -
accuracy: 0.9920 - loss: 0.4812 - val_accuracy: 0.9920 - val_loss: 0.4812
Epoch 44/50
16/16             0s 10ms/step -
accuracy: 0.9920 - loss: 0.4755 - val_accuracy: 0.9920 - val_loss: 0.4755
```

```
Epoch 45/50
16/16             0s 10ms/step -
accuracy: 0.9920 - loss: 0.4698 - val_accuracy: 0.9920 - val_loss: 0.4699
Epoch 46/50
16/16             0s 10ms/step -
accuracy: 0.9920 - loss: 0.4640 - val_accuracy: 0.9920 - val_loss: 0.4642
Epoch 47/50
16/16             0s 10ms/step -
accuracy: 0.9920 - loss: 0.4584 - val_accuracy: 0.9920 - val_loss: 0.4586
Epoch 48/50
16/16             0s 11ms/step -
accuracy: 0.9920 - loss: 0.4527 - val_accuracy: 0.9900 - val_loss: 0.4531
Epoch 49/50
16/16             0s 11ms/step -
accuracy: 0.9920 - loss: 0.4471 - val_accuracy: 0.9900 - val_loss: 0.4475
Epoch 50/50
16/16             0s 10ms/step -
accuracy: 0.9920 - loss: 0.4415 - val_accuracy: 0.9900 - val_loss: 0.4420
```





[17]:
```
conv_filter=mode_avg.get_weights()[0]
my_kernel = conv_filter[:, :, 0, 0]
plt.imshow(my_kernel, cmap='gray')
plt.colorbar()
plt.title("Learned Kernel with relu activation")
plt.show()
```



With average pooling, we have the same accuracy of 1 on the validation set but the kernel's appearence changed and th accuracy reaches 1 more quickly.